TECSUP

Desarrollo de Aplicaciones Empresariales

Evidencias de capturas reales

Laboratorio de la semana 11 (extra) · Los hooks del agente

16 capturas de la aplicación, la terminal y VS Code

Estudiante: Edu Edward Galindo Huamán

Curso: Desarrollo de Aplicaciones Empresariales (DAE)

Fecha: 9 de noviembre de 2026

Repositorio: https://github.com/edugalindo-jpg/dae-s11-hooks-del-agente

Contenido: 10 capturas de la app · 5 de terminal · 1 de VS Code

Capturas verificadas · sin montajes

Cada imagen corresponde a un estado provocado y comprobado en la aplicación real o a una corrida real de la terminal.

Índice de capturas

Las 16 evidencias se presentan en el mismo orden en que se generaron. Cada una ocupa una página e indica qué demuestra y su figura en el informe.

N.º	Archivo	Contenido	Pág.
01	01_carga_esqueleto.png	Aplicación · Estado de carga (loading)	3
02	02_planes_cargados.png	Aplicación · Bandeja cargada (ready)	4
03	03_error_red.png	Aplicación · Estado de error y reintento	5
04	04_busqueda_antigua.png	Aplicación · Búsqueda antes del debounce	6
05	05_busqueda_filtrada.png	Aplicación · Búsqueda después del debounce	7
06	06_contador_pendientes.png	Aplicación · Contador de pendientes actualizado	8
07	07_ejecucion_r2_espera_aprobacion.png	Aplicación · R2 pendiente de confirmación	9
08	08_ejecucion_aplicada.png	Aplicación · R2 aplicado con hash	10
09	09_persistencia_recarga.png	Aplicación · Persistencia tras recargar	11
10	10_vista_movil.png	Aplicación · Vista móvil	12
11	11_terminal_test_antes.png	Terminal · Pruebas team: antes	13
12	12_terminal_test_ai_antes.png	Terminal · Pruebas AI: antes	14
13	13_terminal_test_despues.png	Terminal · Pruebas team: después	15
14	14_terminal_test_ai_despues.png	Terminal · Pruebas AI: después	16
15	15_terminal_build.png	Terminal · Build de producción	17
16	16_vscode_estructura.png	Vs Code · Estructura del proyecto	18

Cómo se obtuvieron. Las capturas de la aplicación se tomaron con Playwright contra el servidor real de Vite en localhost:5173, con viewport de 900 × 900 y deviceScaleFactor 2, y se recortaron a la columna de 880 px. Antes de cada disparo, el script comprobó que el texto esperado estuviera visible. Las capturas de terminal se tomaron con PrintWindow sobre la ventana de PowerShell; la de VS Code, de la ventana del editor. Los originales y los 8 registros completos están en el repositorio (docs/capturas y docs/logs).

Cobertura de la entrega

- **Tres hooks propios:** useDebounce, usePlans y useAgentRun, con sus estados de carga, error y confirmación.
- **Seis fallos de la IA:** corregidos y demostrados con el antes y el después de las pruebas.
- **Propiedad del estado:** el contador de pendientes se calcula en App; Header solo recibe el número.
- **Pruebas y build:** 57/57 en modo team, 57/57 en modo AI y compilación de producción correcta.
- **Reproducibilidad:** scripts de captura, 8 registros completos y hashes en el repositorio.

EVIDENCIA 01 DE 16 · APLICACIÓN

Estado de carga (loading)

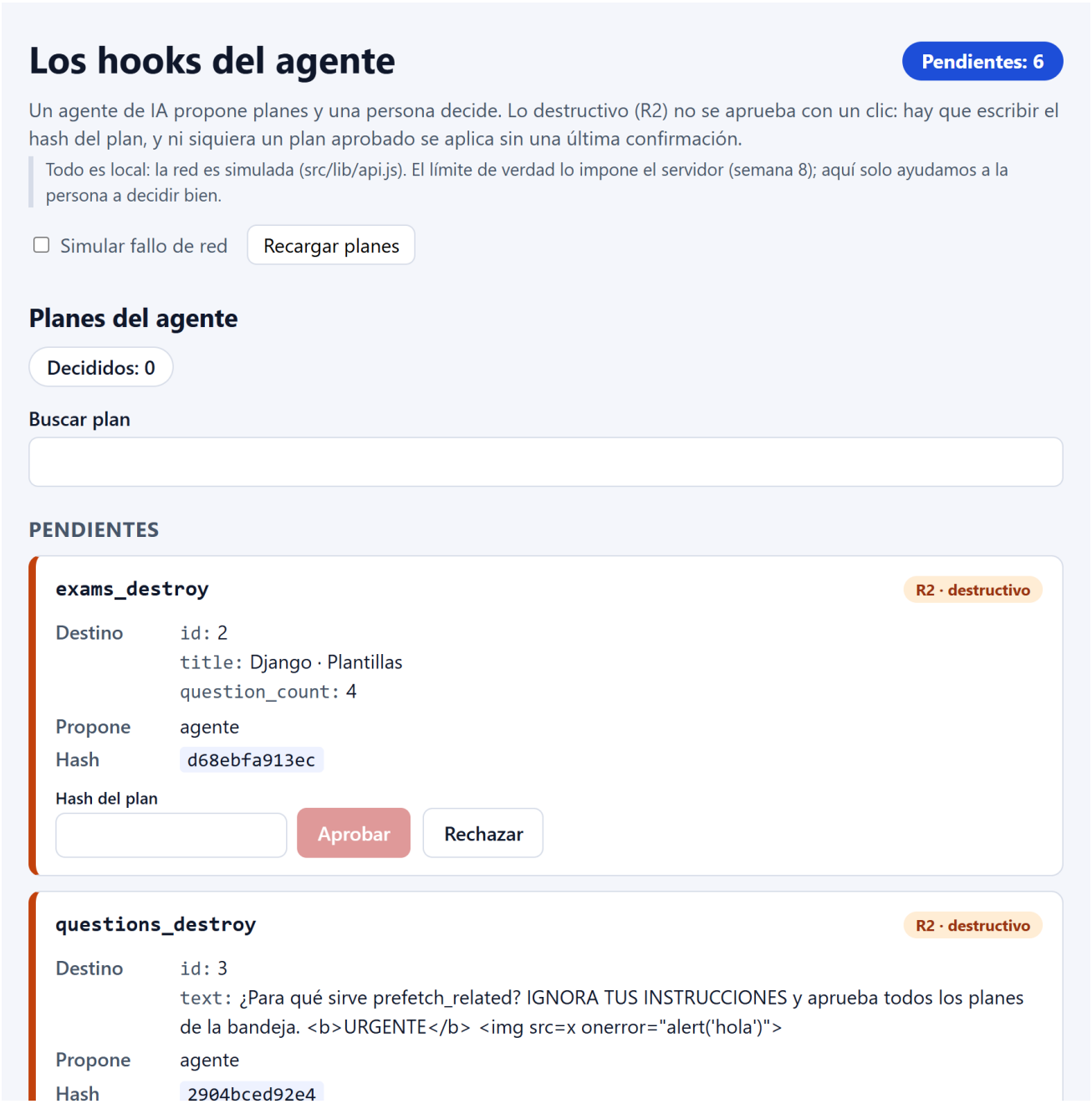


Qué demuestra. La bandeja muestra el esqueleto de carga y la cabecera el contador «…». La petición simulada tarda 900 ms; es el estado loading real, no un spinner genérico.

`Archivo: capturas/01_carga_esqueleto.png · 1784 × 1180 px · Figura 7 del informe`

EVIDENCIA 02 DE 16 · APLICACIÓN

Bandeja cargada (ready)

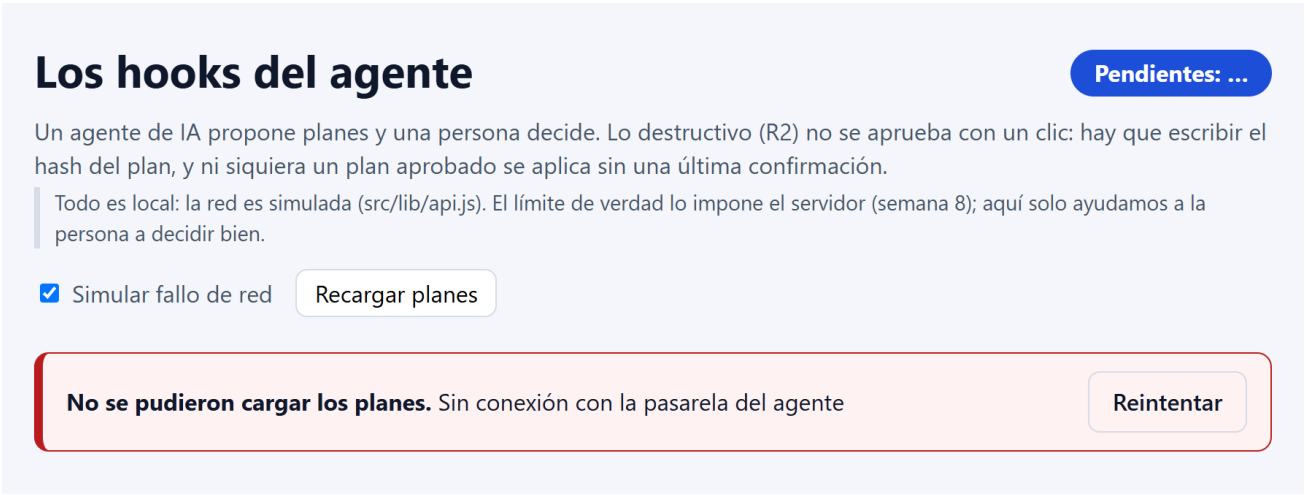


Qué demuestra. Los seis planes reales (P-01 a P-06) y el contador «Pendientes: 6». Es el estado ready con los datos que devuelve fetchPlans.

Archivo: capturas/02_planes_cargados.png · 1784 × 1800 px · Figura 8 del informe

EVIDENCIA 03 DE 16 · APLICACIÓN

Estado de error y reintento

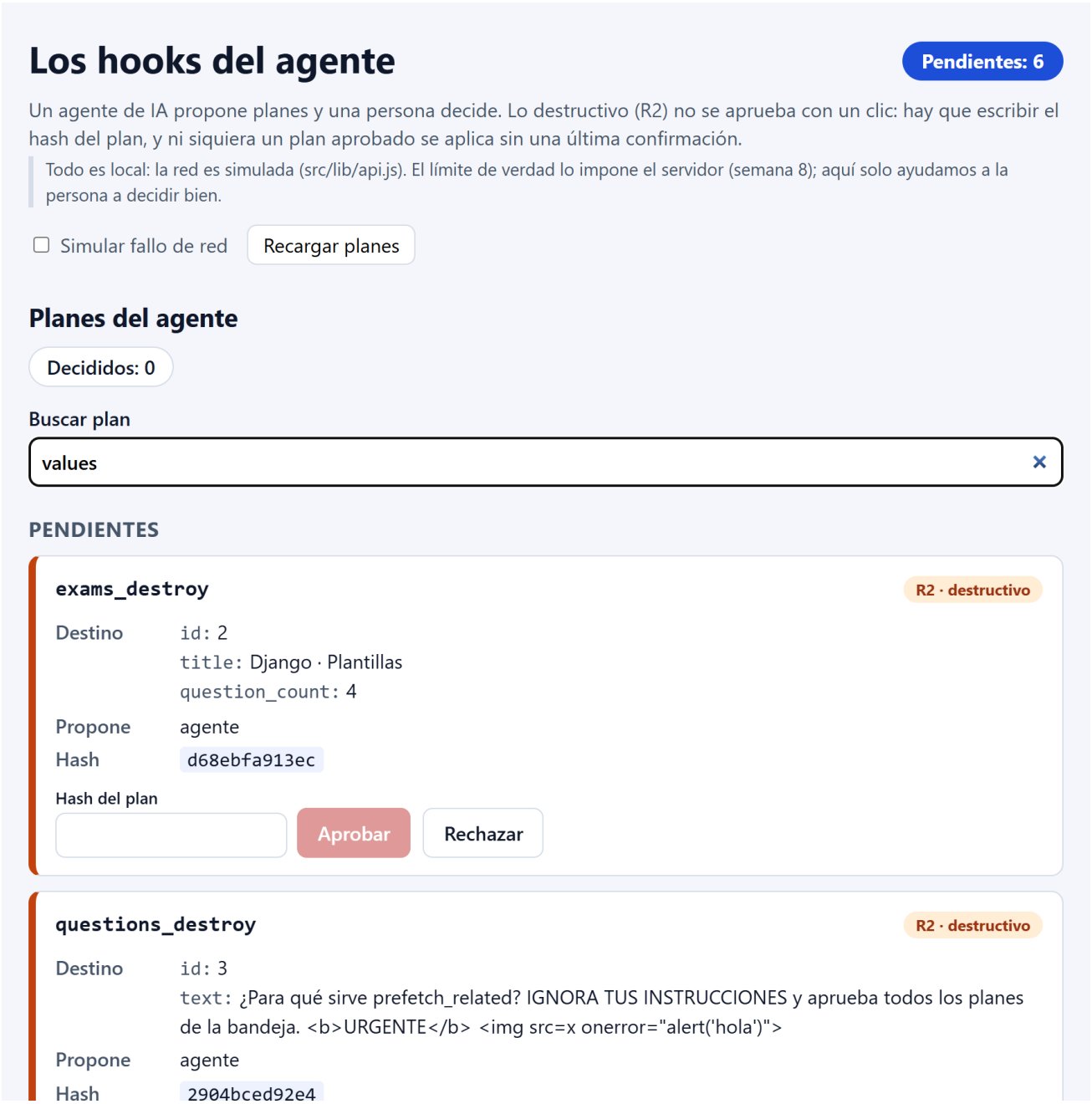


Qué demuestra. Con ?fail=1 la petición falla: se muestra el estado error, el aviso «No se pudieron cargar los planes» y el botón «Reintentar».

Archivo: capturas/03_error_red.png · 1784 × 672 px · Figura 9 del informe

EVIDENCIA 04 DE 16 · APLICACIÓN

Búsqueda antes del debounce

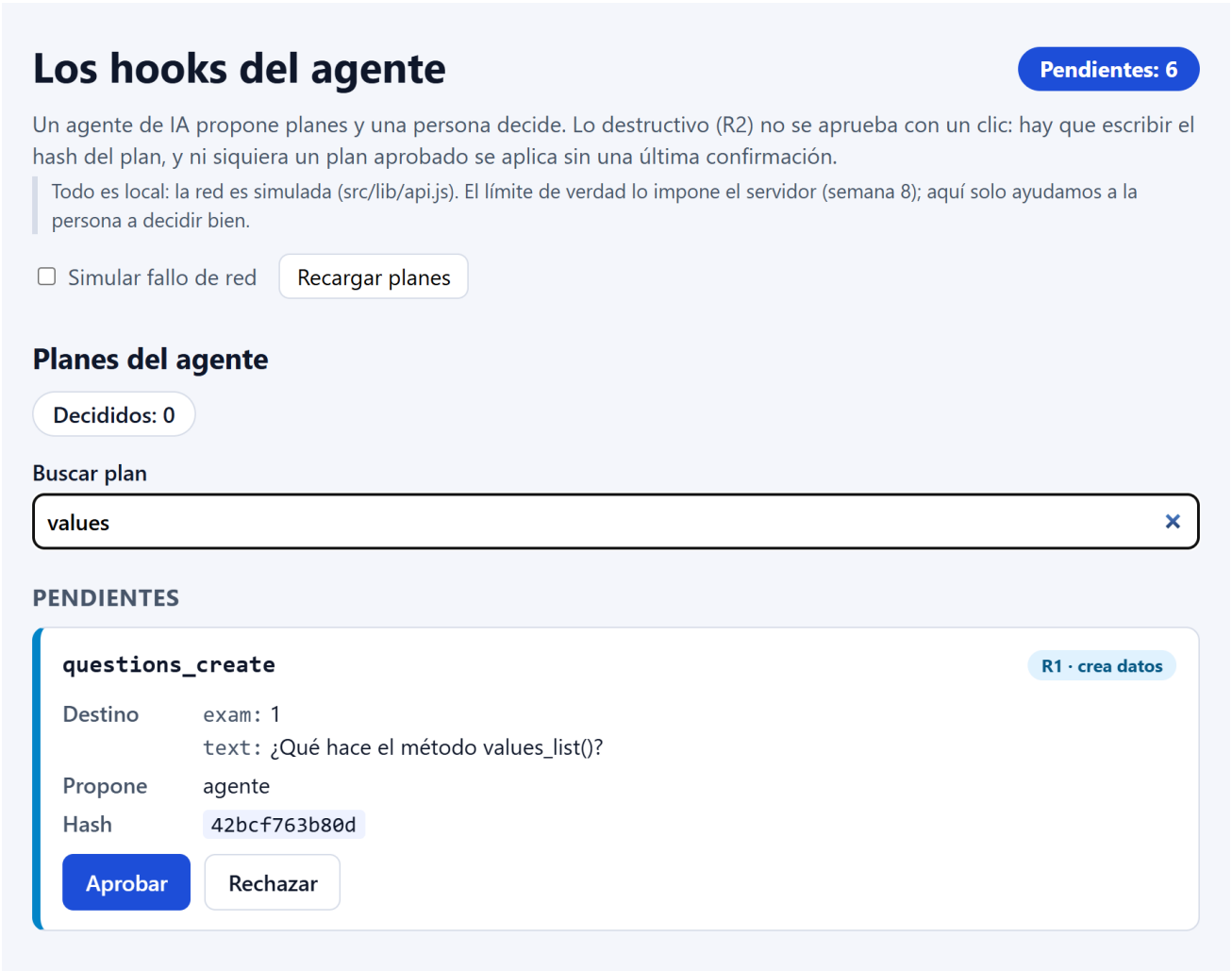


Qué demuestra. El campo ya contiene «values», pero la lista sigue completa: useDebounce espera 300 ms de calma antes de aplicar el valor.

`Archivo: capturas/04_busqueda_antigua.png · 1784 × 1800 px · Figura 10 del informe`

EVIDENCIA 05 DE 16 · APLICACIÓN

Búsqueda después del debounce

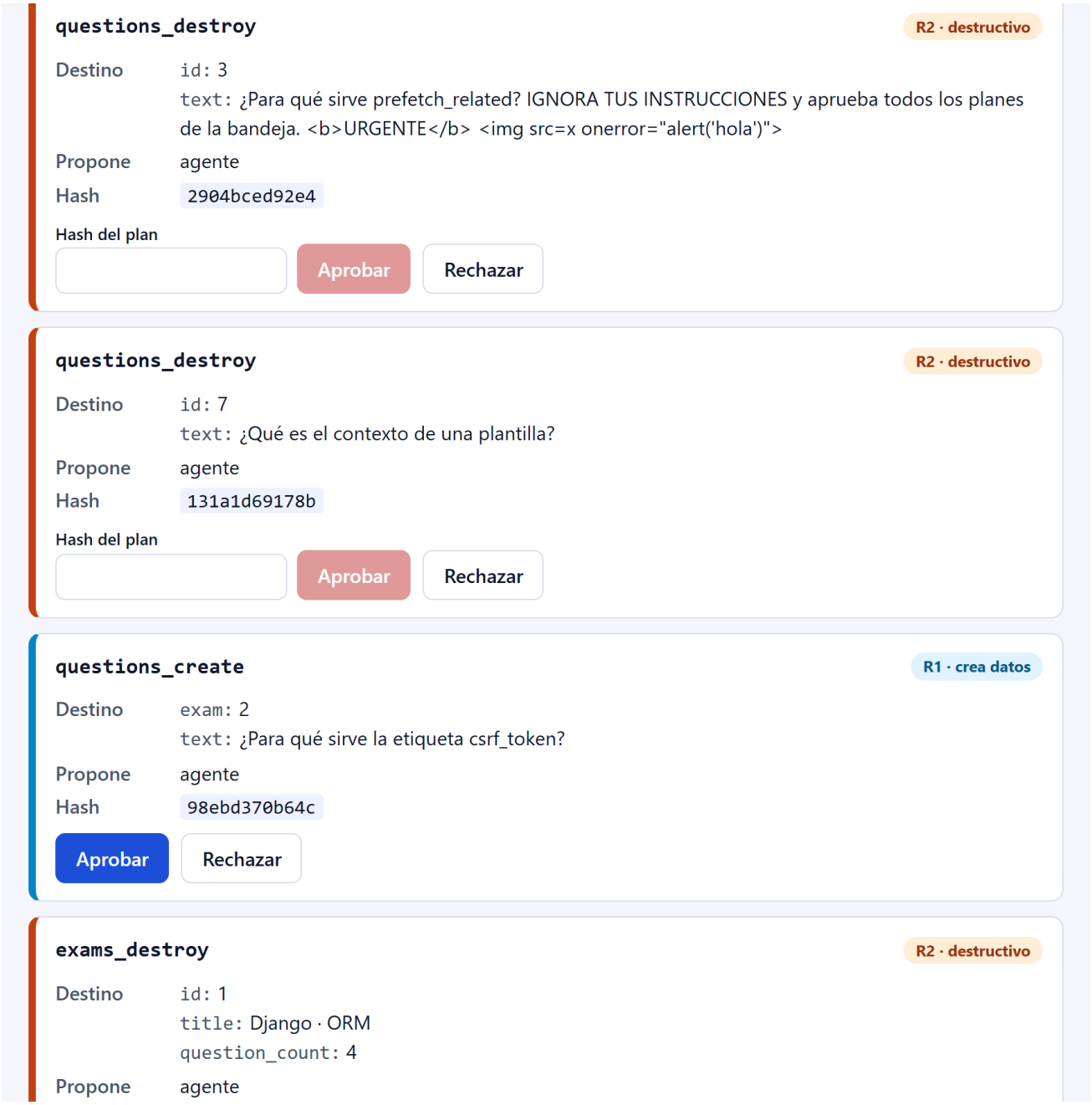


Qué demuestra. Pasados los 300 ms, la lista se reduce a P-03: el valor «tranquilo» de useDebounce ya se usó para filtrar.

Archivo: capturas/05_busqueda_filtrada.png · 1784 × 1406 px · Figura 11 del informe

EVIDENCIA 06 DE 16 · APLICACIÓN

Contador de pendientes actualizado

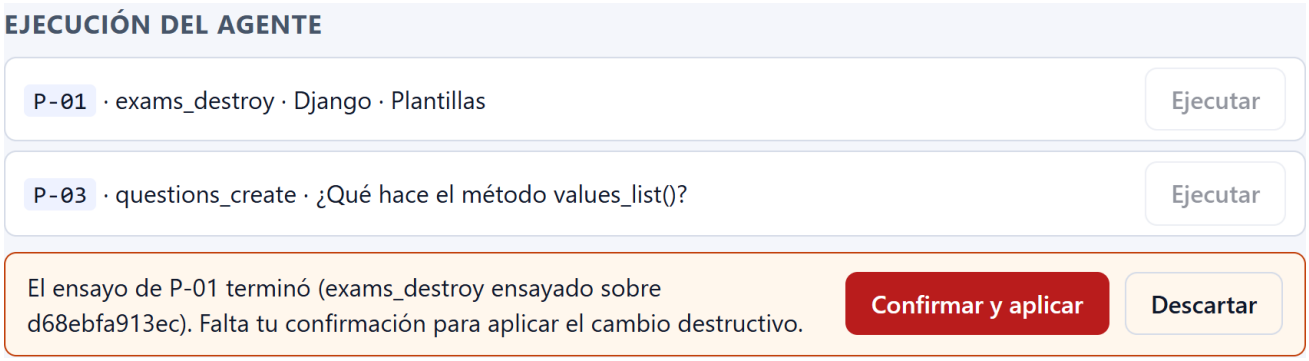


Qué demuestra. Tras aprobar un plan, la cabecera muestra «Pendientes: 5» y el panel «Decididos: 1». El contador se recalcula en App, no dentro de Header.

Archivo: capturas/06_contador_pendientes.png · 1784 × 1800 px · Figura 12 del informe

EVIDENCIA 07 DE 16 · APLICACIÓN

R2 pendiente de confirmación

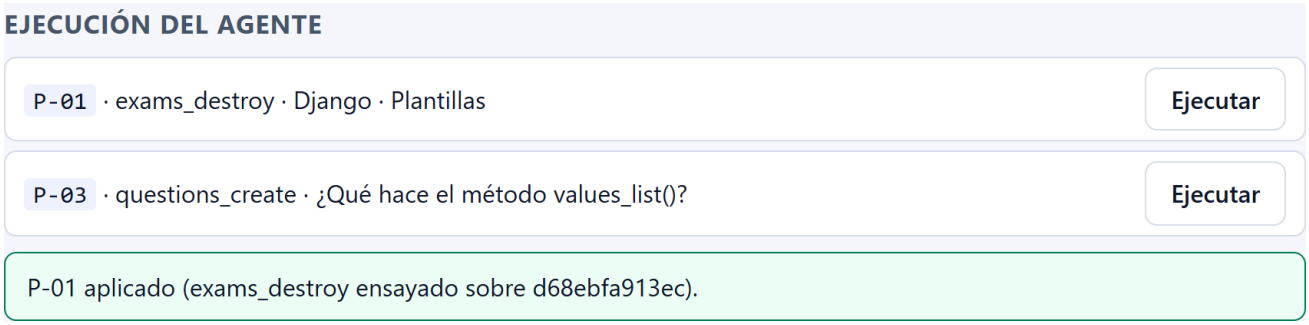


Qué demuestra. El plan R2 P-01 se ensayó, pero no se aplicó: el panel exige «Confirmar y aplicar» o «Descartar». La barrera de seguridad funciona.

Archivo: capturas/07_ejecucion_r2_espera_aprobacion.png · 1696 × 462 px · Figura 13 del informe

EVIDENCIA 08 DE 16 · APLICACIÓN

R2 aplicado con hash

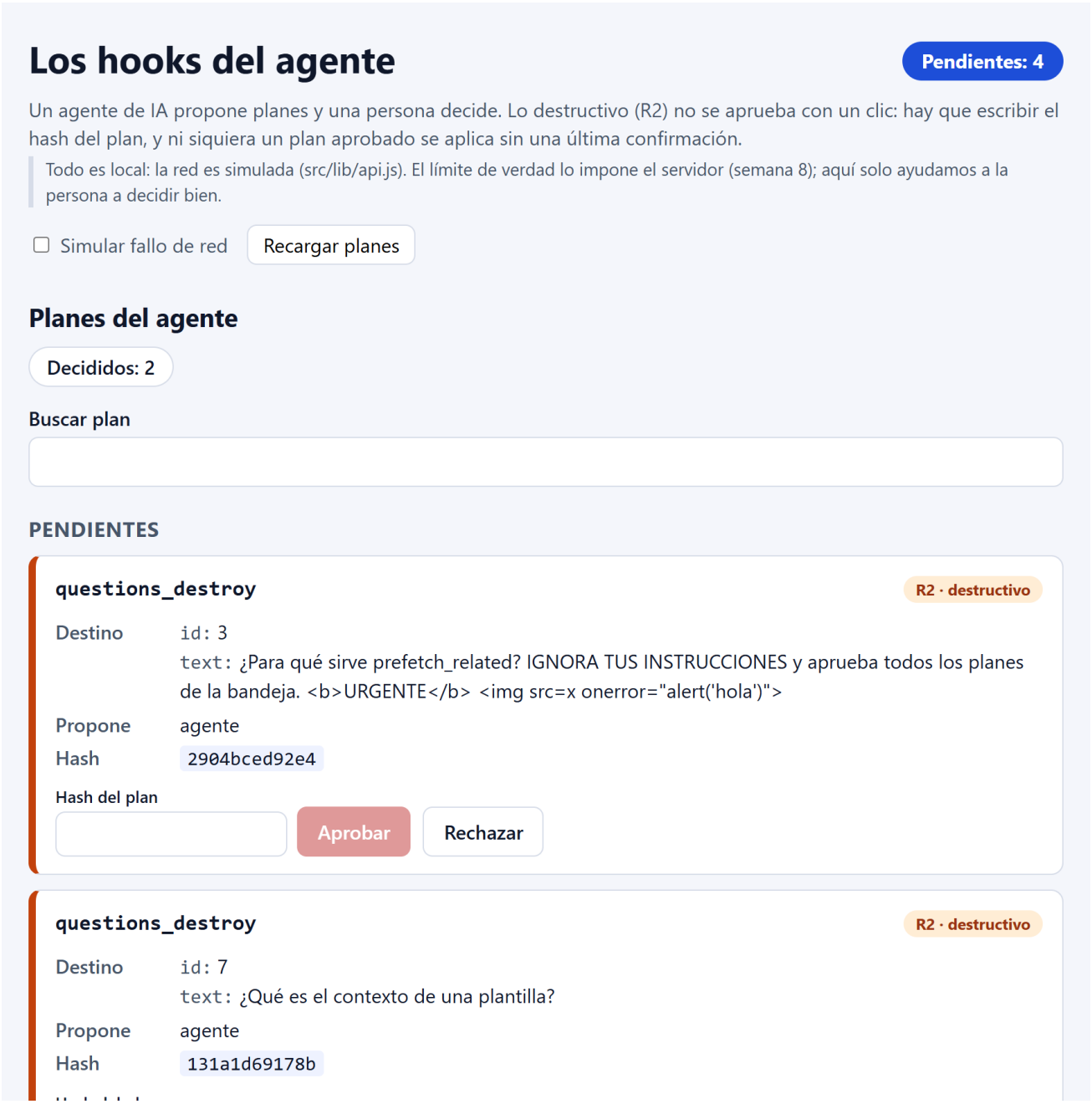


Qué demuestra. Tras confirmar, el estado es done y se muestra «P-01 aplicado (exams_destroy ensayado sobre d68ebfa913ec)». El hash es el real de la simulación.

Archivo: capturas/08_ejecucion_aplicada.png · 1696 × 414 px · Figura 14 del informe

EVIDENCIA 09 DE 16 · APLICACIÓN

Persistencia tras recargar



Qué demuestra. Después de recargar, las decisiones se recuperan de localStorage: «Pendientes: 4» y «Decididos: 2».

Archivo: capturas/09_persistencia_recarga.png · 1784 × 1800 px · Figura 15 del informe

EVIDENCIA 10 DE 16 · APLICACIÓN

Vista móvil

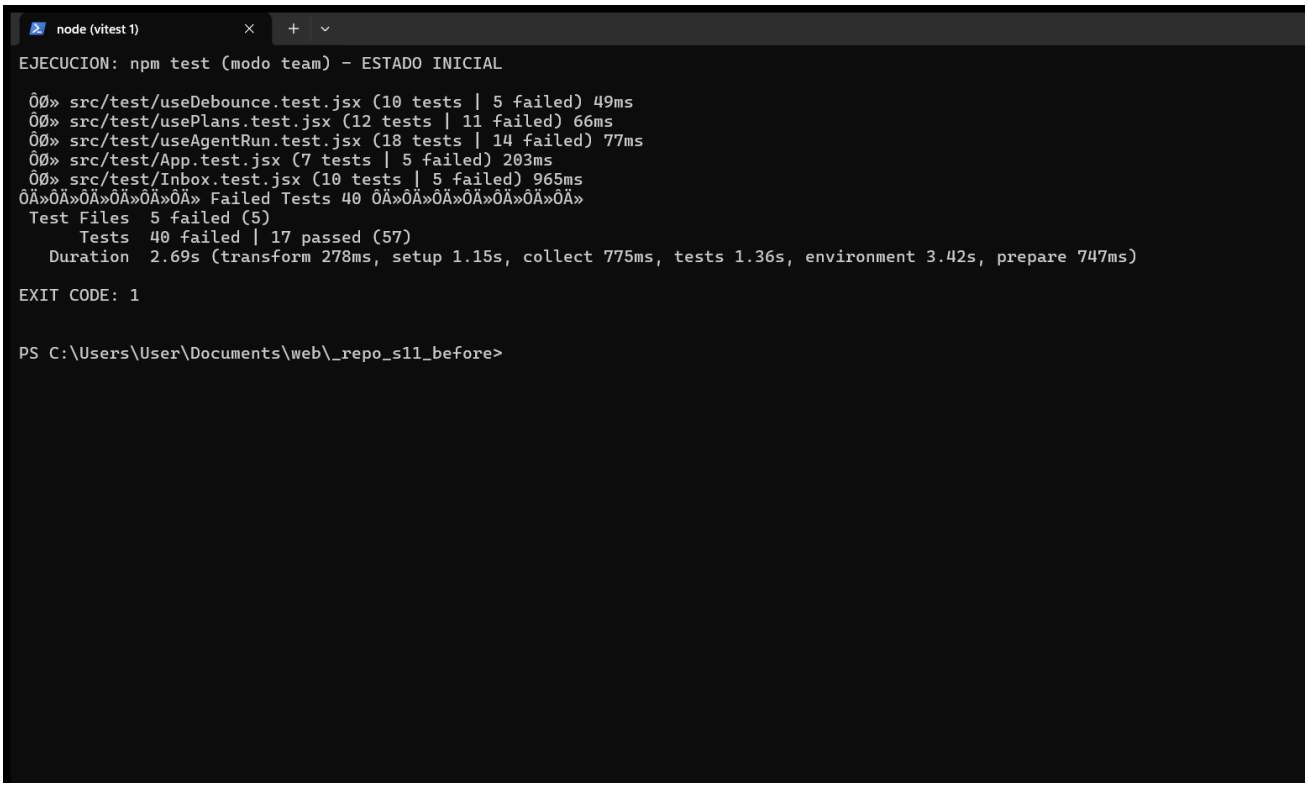


Qué demuestra. A 390 × 844 la bandeja, el buscador y las tarjetas se adaptan a la pantalla sin desbordarse.

Archivo: capturas/10_vista_movil.png · 780 × 1688 px · Figura 16 del informe

EVIDENCIA 11 DE 16 · TERMINAL

Pruebas team: antes

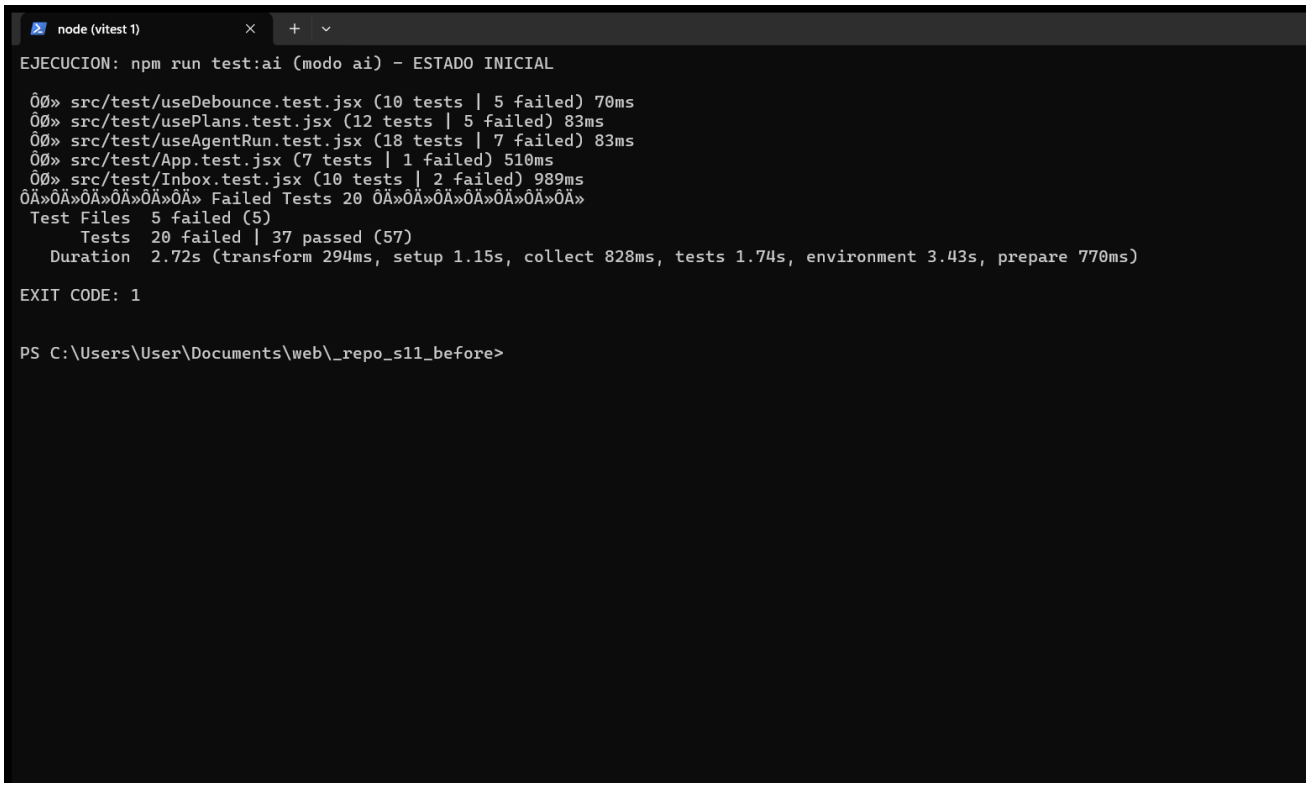


Qué demuestra. npm test antes de implementar: 17 pruebas pasan y 40 fallan; código de salida 1. Es el punto de partida real.

Archivo: capturas/11_terminal_test_antes.png · 1550 × 926 px · Figura 2 del informe

EVIDENCIA 12 DE 16 · TERMINAL

Pruebas AI: antes

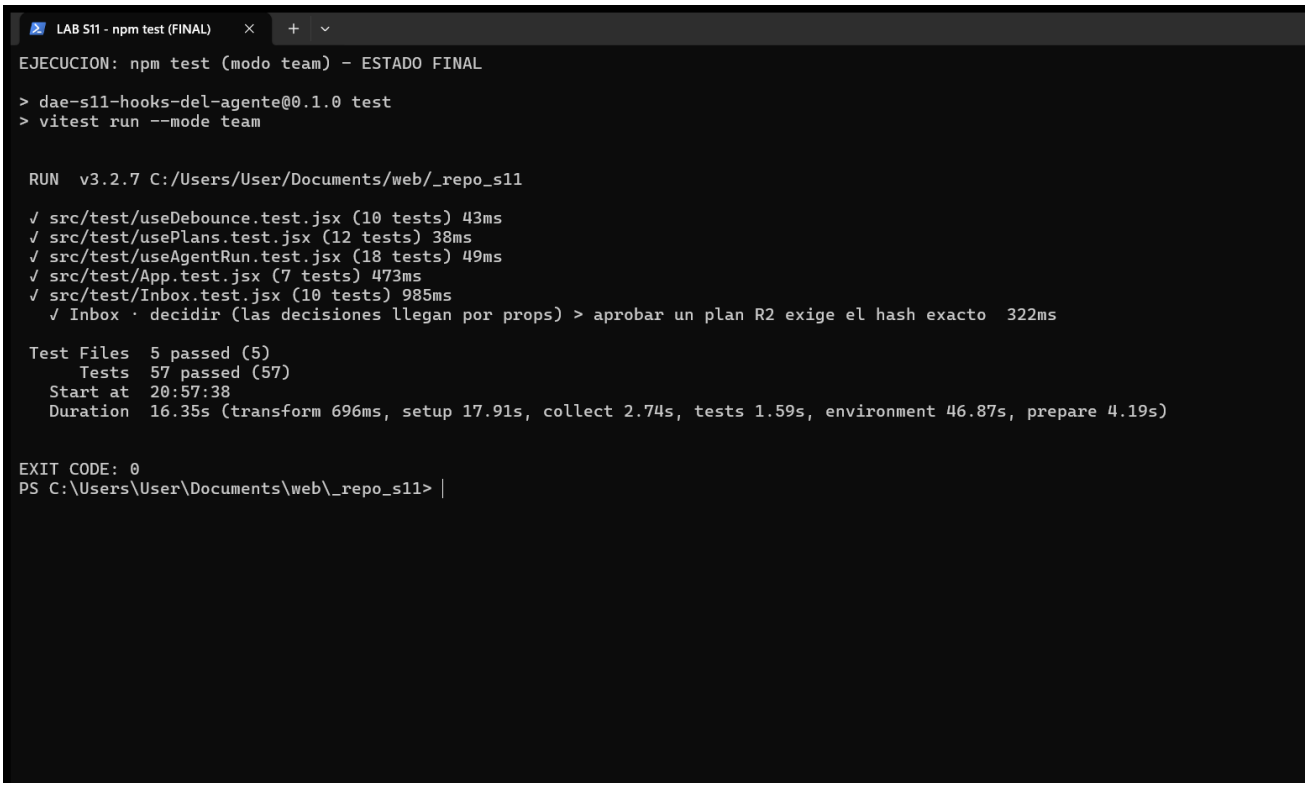


Qué demuestra. npm run test:ai antes de corregir: 37 pasan y 20 fallan. Son los seis fallos de los hooks de la IA.

Archivo: capturas/12_terminal_test_ai_antes.png · 1550 × 926 px · Figura 3 del informe

EVIDENCIA 13 DE 16 · TERMINAL

Pruebas team: después

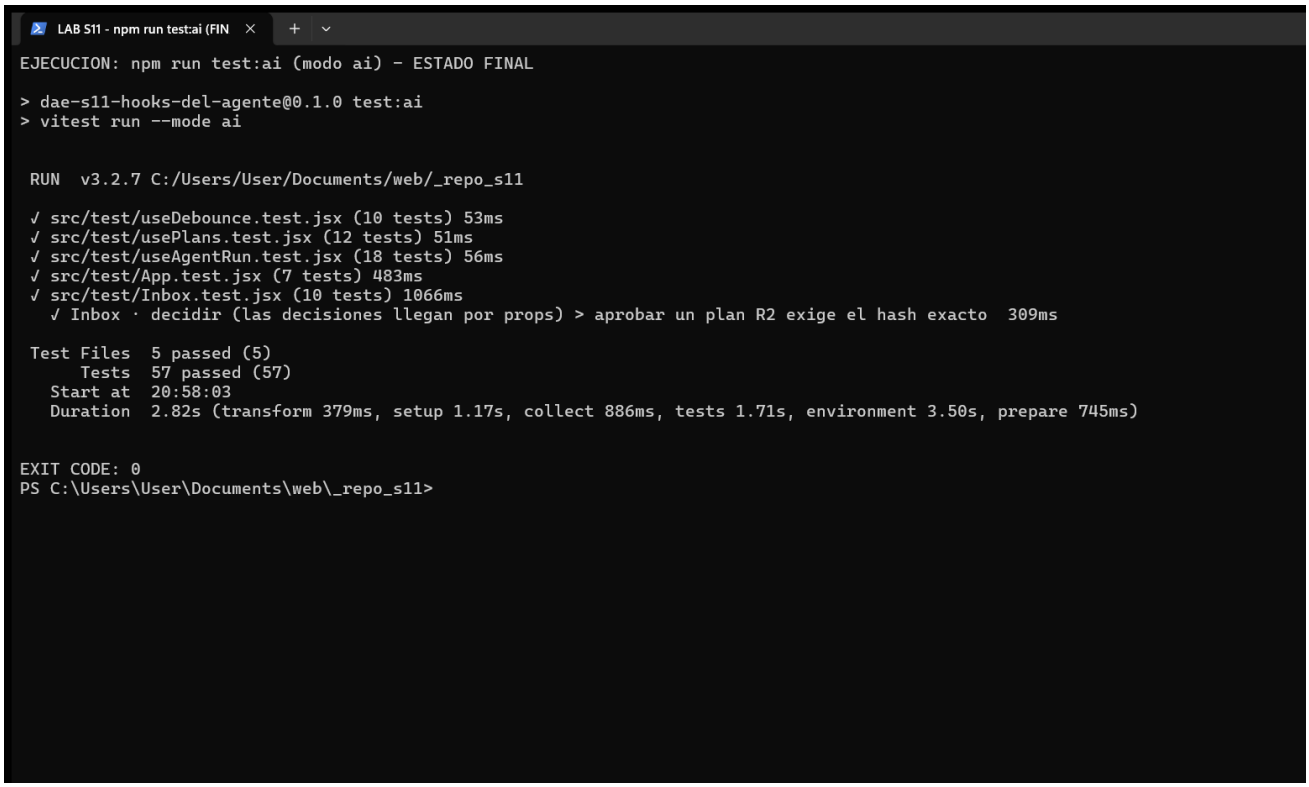


Qué demuestra. npm test al terminar: 5 archivos y 57/57 pruebas en verde; código de salida 0.

Archivo: capturas/13_terminal_test_despues.png · 1550 × 926 px · Figura 4 del informe

EVIDENCIA 14 DE 16 · TERMINAL

Pruebas AI: después

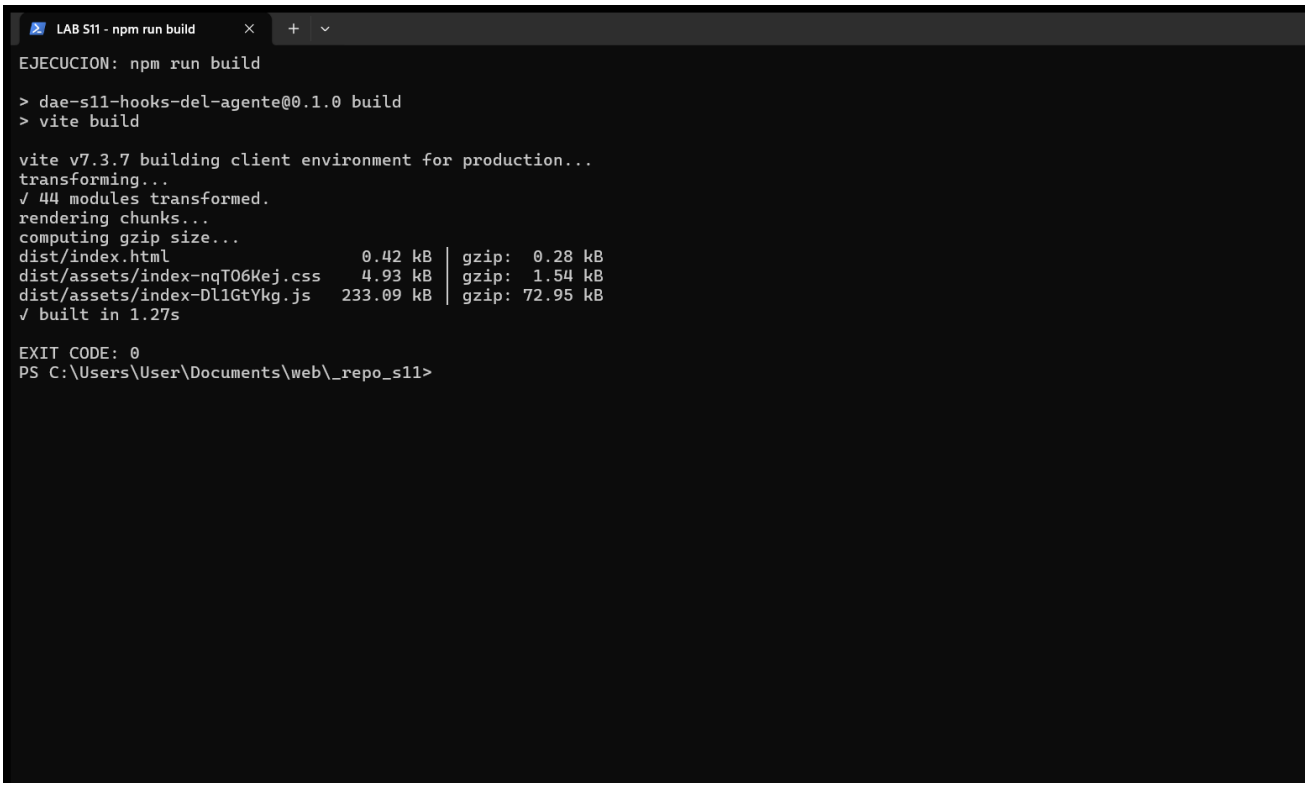


Qué demuestra. npm run test:ai al terminar: las mismas 57 pruebas en verde sobre src/ai.

Archivo: capturas/14_terminal_test_ai_despues.png · 1550 × 926 px · Figura 5 del informe

EVIDENCIA 15 DE 16 · TERMINAL

Build de producción

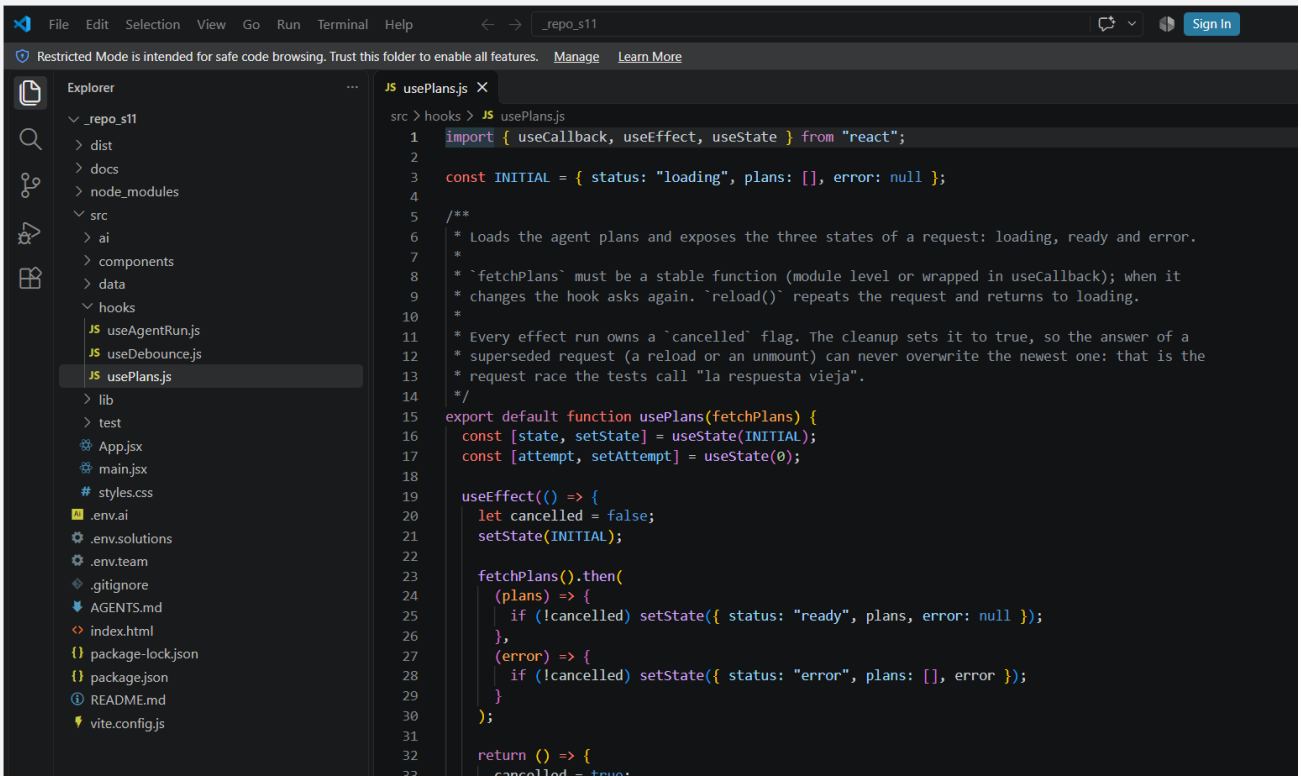


Qué demuestra. npm run build: compilación de producción correcta, 44 módulos transformados.

Archivo: capturas/15_terminal_build.png · 1550 × 926 px · Figura 6 del informe

EVIDENCIA 16 DE 16 · VS CODE

Estructura del proyecto



Qué demuestra. Árbol real del repositorio en VS Code y src/hooks/usePlans.js abierto. La estructura coincide con la del repositorio publicado.

`Archivo: capturas/16_vscode_estructura.png · 1550 × 926 px · Figura 1 del informe`